DESIGN THINKING – 5 STAGE PROCESS

▪ Stage 1: Empathize—Research Your Users' Needs

▪ Stage 2: Define—State Your Users' Needs and Problems

▪ Stage 3: Ideate—Challenge Assumptions and Create Ideas

▪ Stage 4: Prototype—Start to Create Solutions

▪ Stage 5: Test—Try Your Solutions Out

PROBLEM IDENTIFICATION – STEPS TO FOLLOW

▪ Identify a problem that needs to be solved by implementing an IT solution.

▪ The scope should not be too small.

▪ List down the main features you need to implement.

▪ Prepare a simple document indicating the following;

– Problem identified

– Solution

– Features

FAMOUS IDEA GENERATION TECHNIQUES

- Brainstorming
- Mind mapping

Mind mapping is a graphical technique for imagining connections between various pieces of information or ideas.

WHAT IS IDEA (Problem) VALIDATION

▪ Idea validation is the process of gathering evidences to justify the existence of the problem

How to validate the idea

• Literature Survey (find available publications)

– News paper articles

– Websites – Journals

– Books

– Magazines

– TV story

• How to find literature

– Google.com

– scholar.google.com

Internal Testing functional: Black box testing

White Box testing

External Unit Testing: User acceptance testing (UAT)

SDLC (Software Development Life Cycle)

WHAT IS SDLC MODEL?

❑ A framework that describes the activities performed at each stage of a software development project.

❑ A software development life cycle (SDLC) model is a conceptual framework describing all activities in a software development project from planning to maintenance

1. WATERFALL MODEL
2. STRUCTURED EVOLUTIONARY PROTOTYPING MODEL
3. INCREMENTAL SDLC MODEL
4. SPIRAL SDLC MODEL

WATERFALL MODEL

❑ Requirements – defines needed information, function, behavior, performance and interfaces.

- Functional – UI (user interface)
- Non – functional security / reliability / availability

❑ Design – data structures, software architecture, interface representations, algorithmic details.

- ER diagram is a example of a designing software

❑ coding – source code, database, user documentation, testing.

❑ Test

❑ Installation / Maintenance

WATERFALL STRENGTHS

- ❑ Easy to understand, easy to use
- ❑ Provides structure to inexperienced staff
- ❑ Milestones are well understood
- ❑ Sets requirements stability
- ❑ Good for management control (plan, staff, track)
- ❑ Works well when quality is more important than cost or schedule

WATERFALL DEFICIENCIES

- ❑ All requirements must be known upfront
- ❑ Deliverables created for each phase are considered frozen – inhibits flexibility
- ❑ Can give a false impression of progress
- ❑ Does not reflect problem-solving nature of software development – iterations of phases
- ❑ Integration is one big bang at the end
- ❑ Little opportunity for customer to preview the system (until it may be too late)

WHEN TO USE THE WATERFALL MODEL

- ❑ Requirements are very well known
- ❑ Product definition is stable
- ❑ Technology is understood
- ❑ New version of an existing product
- ❑ Porting an existing product to a new platform

STRUCTURED EVOLUTIONARY PROTOTYPING MODEL

❑ Developers build a prototype during the requirements phase

❑ Prototype is evaluated by end users

❑ Users give corrective feedback

❑ Developers further refine the prototype

❑ When the user is satisfied, the prototype code is brought up to the standards needed for a final product.

STRUCTURED EVOLUTIONARY PROTOTYPING STEPS

❑ A preliminary project plan is developed

❑ An partial high-level paper model is created

❑ The model is source for a partial requirements specification

❑ A prototype is built with basic and critical attributes

❑ The designer builds

- ▪ the database
- ▪ user interface
- ▪ algorithmic functions

❑ The designer demonstrates the prototype, the user evaluates for problems and suggests improvements.

❑ This loop continues until the user is satisfied.

STRUCTURED EVOLUTIONARY PROTOTYPING STRENGTHS

❑ Customers can “see” the system requirements as they are being gathered

❑ Developers learn from customers

❑ A more accurate end product

❑ Unexpected requirements accommodated

❑ Allows for flexible design and development

❑ Steady, visible signs of progress produced

❑ Interaction with the prototype stimulates awareness of additional needed functionality

STRUCTURED EVOLUTIONARY PROTOTYPING WEAKNESSES

❑ Tendency to abandon structured program development for “code-and-fix” development

❑ Bad reputation for “quick-and-dirty” methods

❑ Overall maintainability may be overlooked

❑ The customer may want the prototype delivered.

❑ Process may continue forever (scope creep)

WHEN TO USE STRUCTURED EVOLUTIONARY PROTOTYPING

❑ Requirements are unstable or have to be clarified.

❑ As the requirements clarification stage of a waterfall model.

❑ Develop user interfaces.

❑ Short-lived demonstrations.

❑ New, original development

INCREMENTAL SDLC MODEL

❑ Construct a partial implementation of a total system

❑ Then slowly add increased functionality

❑ The incremental model prioritizes requirements of the system and then implements them in groups.

❑ Each subsequent release of the system adds function to the previous release, until all designed functionality has been implemented.

INCREMENTAL MODEL STRENGTHS

❑ Develop high-risk or major functions first

❑ Each release delivers an operational product

❑ Customer can respond to each build

❑ Uses “divide and conquer” breakdown of tasks

❑ Lowers initial delivery cost

❑ Initial product delivery is faster

❑ Customers get important functionality early

❑ Risk of changing requirements is reduced

INCREMENTAL MODEL WEAKNESSES

❑ Requires good planning and design

❑ Requires early definition of a complete and fully functional system to allow for the definition of increments

❑ Well-defined module interfaces are required (some will be developed long before others)

❑ Total cost of the complete system is not lower

WHEN TO USE THE INCREMENTAL MODEL

❑ Risk, funding, schedule, program complexity, or need for early realization of benefits.

❑ Most of the requirements are known up-front but are expected to evolve over time

❑ A need to get basic functionality to the market early

❑ On projects which have lengthy development schedules

❑ On a project with new technology

SPIRAL SDLC MODEL

❑ Adds risk analysis, and 4gl RAD prototyping to the waterfall model

❑ Each cycle involves the same sequence of steps as the waterfall process model

SPIRAL QUADRANT

❑ Objectives: functionality, performance, hardware/software interface, critical success factors, etc.

❑ Alternatives: build, reuse, buy, sub-contract, etc.

❑ Constraints: cost, schedule, interface, etc

❑ Study alternatives relative to objectives and constraints

❑ Identify risks (lack of experience, new technology, tight schedules, poor process, etc.

❑ Resolve risks (evaluate if money could be lost by continuing system development

SPIRAL QUADRANT

DEVELOP NEXT-LEVEL PRODUCT

❑ Typical activities:

▪ Create a design

▪ Review design

▪ Develop code

▪ Inspect code

▪ Test product

SPIRAL QUADRANT

PLAN NEXT PHASE

❑ Develop project plan

❑ Develop configuration management plan

❑ Develop a test plan

❑ Develop an installation plan

SPIRAL MODEL STRENGTHS

❑ Provides early indication of insurmountable risks, without much cost

❑ Users see the system early because of rapid prototyping tools

❑ Critical high-risk functions are developed first

❑ The design does not have to be perfect

❑ Users can be closely tied to all lifecycle steps

❑ Early and frequent feedback from users

❑ Cumulative costs assessed frequently

SPIRAL MODEL WEAKNESSES

❑ Time spent for evaluating risks too large for small or low-risk projects

❑ Time spent planning, resetting objectives, doing risk analysis and prototyping may be excessive

❑ The model is complex ❑ Risk assessment expertise is required

❑ Spiral may continue indefinitely

❑ Developers must be reassigned during non-development phase activities

❑ May be hard to define objective, verifiable milestones that indicate readiness to proceed through the next iteration

WHEN TO USE SPIRAL MODEL

❑ When creation of a prototype is appropriate

❑ When costs and risk evaluation is important

❑ For medium to high-risk projects

❑ Long-term project commitment unwise because of potential changes to economic priorities

❑ Users are unsure of their needs

❑ Requirements are complex

❑ New product line

❑ Significant changes are expected (research and exploration)

CONTEXT DIAGRAM

❑ Another name for a Context Diagram is a Context-Level Data-Flow Diagram or a Level-0 Data Flow Diagram

What is entity?

- A single unique object is called entity

BENEFITS OF A CONTEXT DIAGRAM

❑ Shows the scope and boundaries of a system at a glance including the other systems that interface with it.

❑ No technical knowledge is assumed or required to understand the diagram.

❑ Easy to draw and amend due to its limited notation.

❑ Easy to expand by adding different levels of DFDs.

❑ Can benefit a wide audience including stakeholders, business analyst, data analysts, developers etc

PROTOTYPE DESIGNING

UI Design

- User Interface Design
- A user interface is the point of interaction between the user and a digital device
- Designing the look and feel, the presentation and interactivity of a product. including buttons, icons, spacing, typography, color schemes, and responsive design.

UX Design

- User Experience Design
- UX design is NOT about visuals
- Focuses on how the overall experience makes the user feel, and how easy it is for the user to accomplish their desired tasks.
- Create easy, efficient, relevant, and all-round pleasant experiences for the user.

Low Fidelity Representation

- Rough representations of concepts that help to validate the concepts quickly at the beginning of the application design process.
- Main tools used in this stage are:
    - Sketches – Freehand drawing
    - Wireframes – Screen blueprint/ Skeletal framework

Medium Fidelity Representation

- Colors, fonts, images, and logos that will shape the wireframe are added
- Represents the structure of information, visualize the content and demonstrate the basic functionality in a static way
- Main tool used in this stage are:
    - Mockup

High Fidelity Representation

- An enriched mockup with UX pieces, interactions, and animations to illustrate how the application function
- Helps to test the main interactions in a similar way to the final product
- Main tool used in this stage are:
    - Prototype

Why Prototype?

- The best feedback mechanism
- To gain user acceptance on the features
- To check the usability aspect of the interfaces
- To communicate the design concept the developers
- Reusable backbone of the interface

SOFTWARE VALIDATION

Software validation plays an immensely significant role during the software development life cycle (SDLC) and helps the testing and development teams to create a quality product.

FEATURES OF SOFTWARE VALIDATION

- ❑ Basically, checks that the developed software, follows all requirements and specifications.
- ❑ Responsible for software's acceptance by the user.

.